Submitted to: NIST National Cybersecurity Center of Excellence (NCCoE), in response to its concept paper on AI agent identity and authorization.

Public edition. The text is as submitted, except as noted in the editor's note at the end.

Response to NIST NCCoE Concept Paper: "Accelerating the Adoption of Software and AI Agent Identity and Authorization"

Submitted by: David Burnett

Date: April 1, 2026

Capacity: This is a personal submission addressing a fundamental input quality problem that affects every agentic AI deployment regardless of control plane architecture. I am Head of Product at ZeroBiometrics, which has submitted a separate corporate response on the identity and authorization control plane. This submission is in my personal capacity as a practitioner. FactLattice, the knowledge representation model referenced herein, is an independent open-source project that illustrates one of several possible solutions to the input quality problem this submission describes.

Comment submission: AI-Identity@nist.gov

What No AI Control Plane Can Fix

The concept paper describes three enterprise use cases where AI agents deliver significant value: workforce efficiency, security operations, and software development. It then identifies the identity and authorization infrastructure — identification, authentication, authorization, delegation, audit, and prompt injection defense — needed to make those use cases safe and governable. This is the right structure: security in service of outcomes. But the complete picture requires both layers: enterprises need their ever-expanding organizational knowledge expressed with the precision that AI systems require to act upon without guesswork — and a cryptographically verifiable control plane to enforce it.

Between the desired outcomes and the control infrastructure, there is a layer the concept paper does not address: how does the agent acquire the precise, auditable knowledge it needs to use its authority effectively? The control plane ensures an agent does not exceed its authorization. It does not ensure the agent has the knowledge to produce correct, compliant output within that authorization.

This submission does not propose an identity or authorization system. It explores a complementary problem: how to encode organizational knowledge — task definitions, operating rules, domain constraints, compliance requirements, workflow processes — in a form that enables AI agents to

take consistent, auditable actions within whatever authorization boundaries a control plane establishes. This is not an exercise in information theory. It is the product of hard-won field experience — months of iteration across multiple knowledge domains to determine what works and what does not when providing structured knowledge to AI agents at runtime, as an alternative to the more expensive and specialized approach of encoding that knowledge into model weights via fine-tuning.

The difference between an AI agent working from unstructured instructions and one working from structured knowledge is not incremental — it is categorical. Structured knowledge provides the scaffolding necessary for an AI system to apply its full analytical capacity to the problem at hand. With scaffolding, the AI system's full processing capacity goes toward determining the best course of action consistent with the knowledge, constraints, and operating rules the organization has encoded — instead of expending capacity on orientation and gap-filling. I have observed this across all three of the concept paper's use cases: - **Workforce efficiency (Section 3.1):** AI agents assess and create policy documents, draft formal responses to government RFIs, and analyze regulatory frameworks — under my direction, with my review, on my authority. This submission was produced by an AI agent operating from structured knowledge in approximately 75 minutes of wall-clock collaboration, including iterative review and independent quality scoring by context-clean review agents. - **Security operations (Section 3.2):** Structured knowledge naturally encodes the artifacts security operations depend on — runbooks, escalation procedures, threat classification rules, incident response playbooks, compliance mappings. These are knowledge artifacts where the consequences of ambiguity are acute: an agent that guesses during incident triage is a liability. Structured knowledge with explicit confidence annotations ensures the agent operates from verified procedures or explicitly flags uncertainty for human escalation. - **Software development (Section 3.3):** I operate a complete specification-driven development lifecycle where AI agents are the workforce at every stage — specification authoring, building, testing, security review, deployment, and maintenance. Each agent has bounded scope defined by its specification input.

The benefits of structured knowledge representation go beyond the domains this concept paper contemplates. I applied the same approach to my 83-year-old father's rehabilitation care in a skilled nursing facility. (My father's medical details are shared in this submission with his express permission.) By encoding his medical history, active problems, medications, orders, and risk factors as structured, cross-referenced knowledge, the AI identified critical care gaps that the facility's clinical team had not addressed and enabled me to be a highly persuasive patient advocate. This was a use case of one person, at no incremental cost, with no specialized model, no fine-tuning, and no ML engineering. The same current-generation AI model that drafts policy documents and writes software produced clinically significant care insights — because the knowledge was structured, not because the model was specialized. That is the democratization opportunity: any organization, at any scale, can achieve the velocity and precision benefits of AI by investing in structured knowledge rather than in model customization.

There are many possible strategies for achieving runtime knowledge injection. The one I have created is called FactLattice — a royalty-free, open-source knowledge representation ecosystem published

at https://github.com/740Dave/factlattice. But FactLattice is one implementation of a set of properties that any suitable system would have. First and foremost, it must be written for consumption by an LLM — with explicit scope boundaries, typed relationships between concepts, cross-references that are declared rather than implied, confidence annotations on every claim, and navigable structure that lets the AI agent find precisely what it needs without reading everything. These properties are what eliminate the ambiguity that an LLM would otherwise fill with statistical inference. It must also be human-auditable — renderable in forms that compliance officers, regulators, and domain experts can read and evaluate. It must be cryptographically attestable, time-invariant, and independent of any specific AI model, vendor, or control plane implementation. The remainder of this submission describes the problem in detail, the properties required of a solution, and FactLattice as one concrete implementation — with the understanding that ensuring agents operate from precise, complete, and current knowledge is the must-solve problem, not the exact knowledge injection model used.

What Structured Knowledge Representation Is Not

Two distinctions are worth drawing explicitly, because these approaches are sometimes conflated with structured knowledge representation: - **It is not RAG (Retrieval-Augmented Generation).** RAG retrieves fragments of unstructured text in response to a query. The retrieved fragments carry no structural relationships, no confidence annotations, no cross-references, and no semantic change tracking. RAG answers “what document mentions this topic?” Structured knowledge representation answers “what are the precise rules, how do they relate to each other, and how confident are we in each?” - **It is not fine-tuning.** Fine-tuning encodes knowledge into model weights — opaque, version-specific, vendor-locked. Structured knowledge representation encodes knowledge into inspectable, portable, attestable documents that any model can consume at runtime.

Scope of This Response

This submission addresses how structured knowledge representation is relevant to any system that implements the technical dimensions this concept paper describes. It directly engages: - Focus Area 1 (General Questions) — enterprise use cases, risks, agentic architecture characteristics, standards considerations - Focus Area 5 (Auditing and Non-Repudiation) — semantic change tracking, cryptographic attestation, end-to-end audit chains - All three use cases — with operational evidence from workforce efficiency and software development, architectural analysis for security operations, and a personal healthcare example demonstrating domain breadth

Focus Areas 2 (Identification), 3 (Authentication), 4 (Authorization), and 6 (Prompt Injection) concern the identity and authorization control plane, which is outside the scope of this knowledge representation submission. The observations herein are relevant to any control plane architecture that emerges from the concept paper’s technical submissions.

The Input Quality Problem

The most significant risk in enterprise AI agent deployment is not that agents will exceed their authorization — any control plane responsive to this RFI would address that. It is that agents will produce incorrect output within their authorization because the input — the knowledge they operate from — is incomplete, imprecise, or stale.

When agents encounter gaps in their instructions — ambiguous scope, undefined edge cases, unclear boundaries on permissible decisions — they fill those gaps with statistical inference indistinguishable from grounded output. The resulting errors are fully authorized, fully logged, and completely non-repudiable. The control plane did its job. The output is still wrong.

This is not a deficiency in any particular AI model. It is structural: the more capable the model, the more plausible the incorrect output looks. Better models make the input quality problem harder to detect, not easier.

The control plane is not an error-correction mechanism for incomplete, imprecise, or stale domain knowledge. Knowledge infrastructure and authorization infrastructure are complementary — both necessary, neither sufficient alone.

Structured knowledge eliminates ambiguity at the input layer. The agent either has the knowledge — from the knowledge graph, with explicit confidence — or it explicitly does not, and can say so. This is prevention, not detection. You don't filter garbage at the output; you eliminate it at the input.

Providing structured knowledge also provides the scaffolding necessary for an AI system to apply its full capacity to the problem at hand. Without scaffolding, the agent spends its processing capacity on orientation — figuring out what is being asked, what context is relevant, what is ambiguous. With scaffolding, the AI system's full capacity goes toward analysis. The examples described below — this RFI response, the medical care insights, and the software development lifecycle — all demonstrate the same mechanism: structured knowledge let the AI system work at its ceiling.

This observation is consistent with published findings from AI model developers. Anthropic's system card for Claude Opus 4.6 reports that the same model equipped with an experimental scaffold achieved over twice the performance of their standard scaffold — suggesting that the structured context provided to an AI system can be as consequential for output quality as the model's own capabilities.

What Structured Knowledge Makes Possible

A point that bears emphasis, because it is frequently underestimated: when an AI agent has access to a rich, structured knowledge base, it can generate any output artifact the organization needs — not just the artifact the knowledge was "written for."

Consider an enterprise that encodes its security operations knowledge as structured, cross-referenced documents: incident classification rules, escalation procedures, compliance mappings, threat models, service dependencies, SLA requirements. That single body of knowledge, without modification, enables an AI agent to produce: - A runbook for a specific incident type, tailored to the on-call team's tooling - A post-incident report for executive leadership, with root cause analysis and remediation timeline - A compliance evidence package for an auditor, mapping incident response actions to the relevant control framework - Updated training materials reflecting lessons learned - A risk assessment for a proposed infrastructure change, grounded in historical incident patterns - The delegation policy artifact (tool allowlists, data flow constraints, escalation thresholds) that a control plane needs to govern the security agent itself - A gap analysis comparing the organization's current security posture against industry best practices or regulatory frameworks — identifying risks, recommending enhancements, and prioritizing those recommendations by cost-effectiveness, implementation complexity, or whatever criteria the organization deems important

None of these artifacts require separate knowledge. They are different projections of the same knowledge graph — different views, for different audiences, generated on demand. The knowledge is authored and maintained once. The artifacts are produced as needed by an AI agent that traverses the graph and renders output appropriate to the request.

This is the property that makes structured knowledge representation fundamentally different from documentation, training data, or prompt engineering. Documentation is a fixed artifact that must be manually updated. Training data is frozen in model weights. Prompt engineering provides instructions for a single session. Structured knowledge is a living substrate from which an unbounded variety of artifacts can be instantiated — and every artifact is traceable to the authoritative knowledge that produced it.

The same principle applies across every domain: a software engineering knowledge base produces code, test suites, deployment configurations, PRDs, executive briefings, and all-hands communications. A medical knowledge base produces care plans, insurance appeals, and clinical concern letters. A policy knowledge base produces this RFI response, the scoring rubric that evaluated it, and the change log that tracked every editorial decision. All from the same knowledge, all consistent, all auditable.

Knowledge As Infrastructure, Not Training Data

The current industry approach to domain-specific agent performance is custom model fine-tuning: encode domain knowledge into model weights via training. Microsoft, Oracle, and other professional services firms are actively selling fine-tuning engagements as the path to enterprise AI effectiveness.

Fine-tuning is appropriate for customer-facing products at massive scale — where millions of interactions justify the training investment and latency demands internalized knowledge. But for the use cases this concept paper describes — internal enterprise workflows requiring policy

conformance, audit trails, fast iteration, and regulatory compliance — fine-tuning has four structural limitations:

Time fragility. A fine-tuned model is a snapshot of a business that no longer exists. No business is static. No set of internal policies is static. AI itself is accelerating the pace of change. A fine-tuned model begins degrading the day after training completes. Retraining cycles are expensive, require scarce ML engineering talent, and produce a new snapshot that will also degrade.

Opacity. Model weights cannot be inspected by a compliance officer, presented to an auditor, or diffed between versions. In regulated contexts — SOX Section 404, HIPAA, GDPR Article 22, DFARS/CMMC — auditable documentation of the rules governing automated processes is a compliance requirement, not a preference. When an AI agent IS the internal control, the artifact governing its behavior must be producible on demand. Fine-tuned weights cannot satisfy this requirement.

Accessibility. Structured knowledge representation achieves comparable domain-specific agent performance without ML engineering: a domain expert works conversationally with a current-generation AI model to produce the structured documents. The concept paper's use cases — managing calendars, creating policy documents, automating security operations, developing software — are activities performed by organizations of every size. An approach that requires specialized ML teams and significant professional services investment to achieve domain-specific agent performance will not serve the breadth of enterprises this concept paper addresses.

Vendor lock-in and version fragility. Fine-tuned weights are specific to both the vendor and the model version. An organization that invests in fine-tuning GPT-4o cannot transfer those weights to Claude, Gemini, or even to GPT-5. Every model upgrade potentially invalidates the prior training investment. This is the opposite of the interoperable, standards-based approach the concept paper envisions.

What a Knowledge Representation Standard Requires

Given the input quality problem and the limitations of fine-tuning, what properties must a knowledge representation standard have to serve as the complementary layer to the control plane?

Ambiguity elimination. The representation must be precise enough that an AI agent — including one with no prior context about the organization — can determine exactly what is being asked, what constraints apply, and where its knowledge ends. If the specification is ambiguous, the agent guesses. If the agent guesses, the output is unreliable regardless of how well the control plane governs it.

Machine-authored, human-auditable. The density and structural precision required for ambiguity elimination exceed what human authorship can sustain efficiently. The standard must be designed for AI authorship — a domain expert provides knowledge conversationally, the AI system produces the

structured document. But every document must be fully readable by humans through appropriate rendering — the human reviews, the machine writes.

Time-invariant. Update the knowledge, and agent behavior changes in the next session. No retraining, no deployment pipeline. The knowledge IS the current state of the organization.

Cryptographically attestable. Knowledge artifacts must be signable — proving who authored what knowledge, when, with what reasoning. Combined with a delegation credential from any control plane, this creates a complete chain: who authorized the agent → what the agent was told → what the agent did.

LLM-independent and version-invariant. The same knowledge must work identically across any current-generation AI model — Claude, GPT, Gemini, Llama, or models that do not yet exist. The organization's investment is in the knowledge, not in a specific model's ability to consume it.

Self-auditing. Every claim should carry explicit confidence. Automated analysis should evaluate whether that confidence is warranted, allocating deeper scrutiny to low-confidence knowledge. The knowledge base should actively identify its own weaknesses.

Domain-agnostic. Engineering specifications, legal policies, HR procedures, financial controls, medical records, security runbooks — all must coexist in a unified structure without vocabulary collisions.

Projectable. The same knowledge must be instantiable as any artifact the organization needs — executable agent workflows, source code, compliance documents, executive briefings, training materials, delegation policy artifacts — from a single authoritative source.

These are requirements for any knowledge representation standard that serves this role — not specific to any particular implementation.

Evidence from Practice

This Submission: An Agentic Workflow for Policy Document Assessment

The concept paper's first use case describes enterprise AI agents "assessing and creating policy documents, generating decision recommendations." Consider what an effective agentic workflow for that use case should look like:

1. An agent reads the source document (in this case, a government RFI) and structures it as a knowledge artifact — extracting questions, focus areas, and evaluation criteria.
2. The agent compares the source against existing organizational knowledge (specifications, prior analysis, domain expertise) to identify relevant material and gaps.
3. The agent drafts an initial response organized around the source document's structure.

4. A human subject matter expert reviews, provides feedback, and directs revisions. Each decision is captured with full reasoning context — what was decided, why, what alternatives were considered, and what assumption would warrant reopening the decision.
5. Independent context-clean review agents — with access only to the source document and the draft, no project history or builder context — evaluate the response against a scoring rubric derived solely from the source document.
6. The human applies structural changes based on review findings.
7. The revised document is re-scored to measure improvement.
8. Control points: Corporate counsel reviews for disclosure risk. An authorized signatory approves for release. The document is submitted.

At each stage, the knowledge representation layer defines what the agent should do. At each control point, a delegation credential from whatever control plane the organization uses would gate who can advance the document. The two layers — knowledge and authorization — interlock at every stage.

This is exactly the workflow that produced the submission you are reading.

An independent context-clean agent — with access only to the NIST concept paper and no access to any draft — created a multi-dimensional evaluation rubric. Two additional context-clean agents scored the draft against this rubric. The feedback identified structural weaknesses that were invisible from inside the drafting process. Revisions based on this independent review produced significant improvement with no new intellectual content — only reorganization guided by structured feedback.

Healthcare Advocacy: How Structured Knowledge Made Me a Better Advocate for My Father

As discussed previously, when my 83-year-old father was admitted to a skilled nursing facility for post-operative rehabilitation, I was a concerned family member with no medical training, trying to evaluate whether his care was adequate. I encoded everything I could gather — his medical history, active problems, medications, care orders, risk factors, conversations with staff, and facility communications — as structured, cross-referenced knowledge. The AI, working from this representation, identified care gaps I would never have found on my own, and armed me with the precise clinical language to raise them with facility staff in terms they could not dismiss. - **Unmonitored BP during physical therapy.** My father had a TIA (stroke) in Q4, 2024 and idiopathic post-operative blood pressure lability that was being managed with PRN Hydralazine for systolic readings above 150 mmHg. The facility's BP monitoring protocol ran on a twice-per-shift schedule at variable times — completely independent of the PT schedule. There was no pre-PT or post-PT blood pressure clearance. I knew all of these individual facts — I had lived through the TIA, I was personally verifying that the facility was checking his blood pressure per schedule, and when I asked the AI direct questions about post-surgical physical therapy risks, it provided sound clinical guidance from its training data. But when the AI had access to the entire structured medical picture — neurological history, active medications, vital sign orders, and therapy scheduling encoded as cross-referenced nodes rather than isolated chart entries — it identified a dangerous gap in my father's care,

unprompted, without being asked to look for it: my father, with documented cerebrovascular history, was being sent to strenuous physical therapy sessions with no verification that his blood pressure was within safe limits at the time of exertion. That insight did not come from asking a better question. It came from giving the AI the complete, structured clinical picture and letting it reason across the full graph. - **Opioid regression caused by the facility's own treatment plan.** My father had successfully weaned off oxycodone post-surgery. The facility suddenly doubled PT intensity, causing sufficient upper-extremity stress to require daily opioid resumption — creating a new skilled nursing need that did not exist before the PT increase. Here, structured knowledge corrected an important gap in what I knew: family received knowledge — my father's own description of his injury — characterized it as a rotator cuff problem. When the AI read the actual surgical report encoded in the structured knowledge, it identified the injury as significantly more severe than a simple rotator cuff tear. That clinical precision explained why the doubled PT schedule was causing enough pain to reverse his opioid weaning progress. Cross-referencing the medication history, PT schedule changes, and the actual musculoskeletal pathology — not the family's understanding of it — surfaced this as a facility-created problem, not a patient deterioration.

The chain of precision was striking: unstructured observations from a family member → precise enumeration in structured knowledge → AI-generated clinical insights I could not have produced alone → precise problem articulation back to me in language I could understand and relay → dramatically more effective communication with facility clinical staff, using their terminology, directed at gaps they had not observed. I went from "concerned son with questions" to "advocate presenting specific, cross-referenced clinical findings" — and the facility responded accordingly.

The structured knowledge representation did not replace clinical judgment — it provided the scaffolding for an AI system to reason across my father's full clinical picture in a way that siloed chart documentation could not support, and it gave me the language to act on those insights. That particular care episode has been resolved, but my father's structured knowledge file continues to grow. Every new provider visit, medication change, and clinical observation is encoded. The longer the file is maintained, the more valuable it becomes — not as a record of a single event, but as a living, cross-referenced clinical picture that any AI system can reason over the next time a care question arises.

Personal Software Development: Specification-Driven SDLC

In my personal software development projects — independent of my work at ZeroBiometrics — I operate a complete specification-driven SDLC where AI agents perform every stage:

Stage	Agent Role	Knowledge Dependency
Spec authoring	Structured editor + automated linting	FactLattice format defines valid structure
Build	Independent builder agents receive linted specs as complete	Spec is the builder's only input

Stage	Agent Role	Knowledge Dependency
	requirements	
First bringup	Launch the built component, fix every crash, iterate until it runs	Spec defines expected behavior
Quality gate	Spec-gap analysis, security review, test coverage	Spec-gap analyzer compares implementation against spec
Deploy	Deploy artifacts, verify services respond, update infrastructure docs	Deployment spec defines target configuration
Maintain	Spec densification, corpus health analysis, drift detection	Spec-implementation drift detected and reconciled

Context-rich vs. context-clean agents. A critical architectural insight: not all agents should carry the same context. Builder agents are context-rich — they operate with full project context because they need the whole picture. But review agents (spec-gap analyzer, security reviewer, QA tester, linter) are context-clean — they start with a fresh context containing ONLY the specification and the implementation under review. No project history, no builder's reasoning, no justification for shortcuts.

A context-clean reviewer cannot be biased by the builder's rationale. It reads the specification cold and asks: "does the implementation match?" This is the same structural independence principle as external audit: the auditor reads the standard and compares, without reading your justifications.

This architecture depends directly on knowledge representation quality: the specification must be precise enough that an agent with no prior project context can evaluate compliance against it. If the specification is ambiguous, context-clean review is impossible — the reviewer would need the builder's context to interpret the spec, defeating the independence guarantee. The precision of the knowledge graph is not just a governance property — it is a precondition for independent verification.

Relationship to the Concept Paper's Standards and Focus Areas

The knowledge representation layer described in this submission interacts with several standards the concept paper identifies:

MCP (Model Context Protocol). A knowledge representation standard operates above MCP's protocol layer. MCP defines how agents invoke tools; the knowledge layer defines which tools a specific task requires, what inputs they expect, and what constraints govern their outputs. A knowledge graph can project tool allowlists that map directly to the delegation credentials an MCP-aware enforcement proxy validates — whatever proxy architecture the organization deploys.

OAuth 2.0. Knowledge-derived delegation scopes are projectable into OAuth scope format. A knowledge graph that defines an agent's authorized actions can produce the specific OAuth scopes

a sender-constrained token should carry — derived from the task definition rather than statically assigned.

NGAC (INCITS 565-2020). A knowledge graph with typed relationships, domain-specific attributes, and confidence annotations is a natural complement to NGAC's attribute-based policy model. Knowledge-derived attributes can inform NGAC policy evaluation without requiring the policy engine to parse unstructured instructions.

SP 800-207 (Zero Trust Architecture). The specification-driven agent pattern embodies zero trust principles: per-specification authorization (not session-based trust), context isolation between agents with different trust roles (micro-segmentation), specification-bounded scope (least privilege), and protocol-layer enforcement (the agent cannot bypass the enforcement point). These are zero trust tenets applied to the knowledge layer.

Prompt injection. From the knowledge representation perspective: a specification-driven agent with protocol-layer enforcement has a bounded action space regardless of what a prompt injection attempts — a successful injection cannot cause scope exceedance when enforcement is external to the agent. Additionally, the context-clean review pattern provides structural resilience: review agents that start with a fresh context cannot be influenced by injected content in a builder's context, because they never see that context.

Integration observation for the demonstration project. The demonstration goals this concept paper envisions will be most effectively achieved by combining whatever identity and authorization architecture emerges from the technical submissions with a structured knowledge representation layer that gives governed agents the precise context they need to operate effectively. Specifically: - Delegation credentials that reference versioned knowledge artifacts (by cryptographic hash) create an end-to-end audit chain: who authorized → what was authorized → what was done. - Protocol-layer enforcement of knowledge-derived policy (tool allowlists, data flow constraints, scope boundaries) ensures the knowledge layer's projections are enforced, not merely documented. - Context isolation between agents with different trust roles — enforceable by the control plane, defined by the knowledge layer — enables independent verification without context contamination.

Recommendations

1. Evaluate structured knowledge representation standards alongside identity and authorization standards. The control plane is more effective when it governs agents whose behavior is driven by external, inspectable artifacts. The demonstration project should assess how structured knowledge interacts with delegation credentials.
2. Include the specification-driven agent pattern in use case evaluation. All three proposed use cases are internal enterprise workflows where specification-driven agents outperform fine-tuned agents on auditability, iteration speed, accessibility, and LLM portability.
3. Consider scalability across organization sizes. The concept paper scopes the project to enterprise. But the fastest path to broad adoption of identity and authorization standards is

through approaches that work at any scale. Structured specifications produced conversationally with any current-generation AI model, combined with a protocol-layer enforcement point, make the complete governance model accessible without ML engineering teams or enterprise-scale infrastructure.

FactLattice, the open-source knowledge representation ecosystem referenced in this submission, has been substantiated in practice across six knowledge domains: biometric cryptographic protocols, AI governance, software engineering, software-defined I/O, medical records, and structured care methodology. It is available at https://github.com/740Dave/factlattice and is not bound to any specific control plane implementation. I welcome the NCCoE's review.

David Burnett davidleeburnett@mac.com

Editor's note (public edition)

The text above is as submitted, with one permitted change: the placeholder link "[github.com/placeholder/factlattice]" appeared twice in the submission and was replaced each time with https://github.com/740Dave/factlattice.

1. In "What No AI Control Plane Can Fix" (paragraph beginning "There are many possible strategies"): original wording "[github.com/placeholder/factlattice]".
2. In "Recommendations" (final paragraph, beginning "FactLattice, the open-source knowledge representation ecosystem"): original wording "[github.com/placeholder/factlattice]".